JRs Thread Shop

OPERATOR HANDBOOK

A practical guide to running, maintaining, and extending the JRs Thread Shop storefront. Covers daily operation, common edits, troubleshooting, and a prioritized roadmap from this frontend scaffold to a production e-commerce site.

Generated 2026-05-16 · Frontend scaffold v0.1.0 · Express + EJS + Vanilla JS/CSS

What's in the box right now

- **Server** — Express app (`app.js`) + listener (`server.js`).
- **Pages** — Home, Shop (filter/sort/search), Product detail, Cart, 404.
- **Data** — 12-product seed in `data/products.json`; 12 generated SVG placeholders.
- **Cart** — localStorage-backed with cross-tab sync and a slide-in mini-cart drawer.
- **SEO** — per-page title/description/canonical, Open Graph, Product JSON-LD, dynamic sitemap, robots.txt.
- **Git** — initialized on `main`; `.claude/` and `node_modules/` ignored.

1. RUNNING & STOPPING THE SERVER

On Windows + PowerShell, use the `npm.cmd` wrapper. The `npm` command resolves to `npm.ps1`, which is blocked by PowerShell's default execution policy. `.cmd` isn't subject to that policy.

Day-to-day (auto-reload on save)

```
npm.cmd run dev
```

Then open **http://localhost:3000**. Node's `--watch` flag restarts the server when `app.js`, `server.js`, or anything it requires changes. EJS templates and CSS/JS in `public/` don't need a restart — just refresh the browser.

Plain (no auto-reload)

```
npm.cmd start
```

Stop the server

Press **Ctrl+C** in the terminal running the server. If it didn't shut down cleanly and port 3000 is stuck:

```
Get-NetTCPConnection -LocalPort 3000 |
  ForEach-Object { Stop-Process -Id $_.OwningProcess -Force }
```

Run on a different port

Set `PORT` before launching. The site URL embedded in metadata (canonical, sitemap) also reads from `SITE_URL`:

```
$env:PORT=4000
$env:SITE_URL='http://localhost:4000'
npm.cmd run dev
```

> **Permanent PowerShell fix (optional)**
>
> If you'd rather use `npm` directly (no `.cmd`), run this once: `Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned`. It only affects your user account and is Microsoft's recommended developer-machine setting.

2. ADDING & EDITING PRODUCTS

All product data lives in `data/products.json`. It's loaded once at server boot, so restart the server (or save any file under watch) after editing.

Product schema

```
{
  "id": "midnight-grid",         // URL slug. Must be unique. Lowercase + hyphens.
  "name": "Midnight Grid",        // Display name.
  "price": 34,                    // Whole-dollar number. USD assumed.
  "category": "Men",              // Men | Women | Unisex (drives the listing filter).
  "image": "/images/tees/midnight-grid.svg",  // Path under /public.
  "colors": ["Navy", "Black"],    // First entry is the default selection.
  "sizes":  ["S","M","L","XL"],   // Order shown in the size chip row.
  "description": "...",           // Plain text. Used as meta description + JSON-LD.
  "tags": ["minimal","everyday"], // Free-form. Searchable on /shop.
  "featured": false,              // true -> shows on home page (max 6).
  "createdAt": "2026-01-10",      // ISO date. Drives the "Newest" sort.
  "stock": 60                     // <20 shows a "Low stock" badge on cards.
}
```

Add a new product

- Append a new object to the array in `data/products.json`.
- Pick a unique `id` — this becomes the URL: `/product/<id>`.
- Drop a matching image at `public/images/tees/<id>.svg` (or .jpg/.png) and point `image` at it.
- Restart `npm.cmd run dev` if it isn't already watching.
- Verify: it appears on **/shop**, opens at **/product/<id>**, and is in **/sitemap.xml**.

Remove a product

Delete the object from the JSON array. The sitemap auto-shrinks. Old URLs will 404 — if Google has indexed them, consider keeping the entry but setting `stock` to 0 instead.

3. IMAGES: SVG PLACEHOLDERS & REAL PHOTOS

Regenerate the SVG placeholders

The bundled SVGs are produced by a one-off script. Re-run it after editing colors or graphics in the generator (or after editing the first color of any product, since that drives the SVG fill):

```
node scripts\generate-tees.js
```

Output: 12 files in `public/images/tees/`, one per product id.

Swap a placeholder for a real photo

- Save the photo as `public/images/tees/<product-id>.jpg` (or .png/.webp).
- Update `image` in `data/products.json` to the new path.
- Use a square 1:1 image — cards and the detail page lock to `aspect-ratio: 1/1`.
- Target ~1200×1200 px, <200 KB. Use WebP for the smallest size.
- Add an `alt` override if you want product-specific phrasing — for now alt = product name.

Multiple images per product (future)

Not wired yet. When you're ready, change `image` from a string to an `images: []` array, update the product card to show `images[0]`, and add a thumbnail strip + main-image swap on the detail page.

Open Graph preview image

Each page's `og:image` defaults to the product image on detail pages. For home/shop you may want a branded 1200×630 image — drop it at `public/images/og-default.png` and pass `ogImage` into the head partial from those pages.

4. EDITING STYLES & TEMPLATES

Where to change what

Want to change…	Edit…
Color palette / fonts / spacing scale	public/css/base.css (the :root tokens)
Buttons, cards, header, mini-cart, footer	public/css/components.css
Hero, listing layout, product page, cart layout	public/css/pages.css
Page <head> / SEO tags	views/partials/head.ejs
Top nav / cart badge / mobile menu	views/partials/header.ejs
Footer / drawer markup / script tags	views/partials/footer.ejs
Reusable product tile	views/partials/product-card.ejs
A whole page	views/{home,shop,product,cart,404}.ejs

Design tokens

All colors, spacing, type sizes, and breakpoints are CSS custom properties at the top of `base.css`. Change one variable — e.g. `--color-accent` — and every button, badge, hover state, and focus ring follows.

```
/* base.css */
:root {
  --color-accent: #ff3b30;   /* swap brand color here */
  --font-display: 'Archivo Black', Impact, sans-serif;
  --wrap-max: 1200px;
  /* ... */
}
```

Mobile-first breakpoints

All media queries are `min-width`. Base styles target mobile; tablets/desktops layer on top:

- **640px** — small tablet (wrap padding & display sizes increase).
- **768px** — nav switches from drawer to horizontal; product grid goes to 3-up.
- **960px** — shop sidebar appears; cart switches to two-column layout.
- **1100px** — product grid becomes 4-up.

5. GIT WORKFLOW

Daily loop

```
git status
git diff                        # see what changed
git add <paths>                 # stage specific files
git commit -m "message"
git log --oneline -10
```

Prefer staging by path over `git add .` — it keeps you from accidentally committing IDE junk, secrets, or generated files.

Branch for non-trivial changes

```
git checkout -b feature/checkout-stripe
# ... work, commit ...
git checkout main
git merge --no-ff feature/checkout-stripe
```

What's already ignored

- `node_modules/` — reinstall with `npm.cmd install`.
- `.env` / `.env.*` — for the eventual Stripe key, DB URL, etc.
- `.vscode/`, `.idea/`, `.claude/` — per-machine editor settings.
- `*.log`, build artifacts.

Push to a remote (when ready)

```
git remote add origin <url-from-github-or-gitlab>
git push -u origin main
```

> **Tip — line endings on Windows**
>
> Git's `warning: LF will be replaced by CRLF` on commit is normal and harmless — it normalizes line endings. If it's noisy and you want it silent: `git config --global core.autocrlf true`.

6. SEO MAINTENANCE

What's already wired

- Per-page `<title>`, meta description, canonical, robots.
- Open Graph + Twitter card tags on every page.
- **JSON-LD** `Product` **schema** on detail pages — price, availability, image, brand.
- **Dynamic** `/sitemap.xml` rebuilt from the product catalog on each request.
- `/robots.txt` with `Disallow: /cart` (no point indexing it).
- Semantic HTML — one `<h1>` per page, breadcrumb nav on detail, skip-link, focus-visible outlines.

Things to do before launch

- Set `SITE_URL` to your real domain so canonical/OG/sitemap URLs are correct.
- Add a branded 1200×630 OG image at `/public/images/og-default.png`.
- Add Google Search Console — submit `https://<domain>/sitemap.xml`.
- Add a basic `Organization` JSON-LD block to the home page.
- Consider `BreadcrumbList` JSON-LD on the product page (you already render visual breadcrumbs).
- Add structured data testing — `https://search.google.com/test/rich-results`.
- Performance: compress photos, add `<link rel="preload">` for the hero image when you have one.

7. TROUBLESHOOTING

EADDRINUSE: address already in use :::3000

Another process (usually a previous server you forgot to stop) is still bound to port 3000.

```
Get-NetTCPConnection -LocalPort 3000 |
  ForEach-Object { Stop-Process -Id $_.OwningProcess -Force }
```

Or run on a different port for this session: `$env:PORT=4000; npm.cmd run dev`.

npm.ps1 cannot be loaded / not digitally signed

PowerShell execution policy blocks unsigned scripts. Two fixes:

- **Quick:** use `npm.cmd run dev` instead of `npm run dev`.
- **Permanent:** `Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned` (then answer Y).

Pages render unstyled or broken

- Check the browser DevTools Network tab — are `base.css`, `components.css`, `pages.css` returning 200?
- Hard-refresh: **Ctrl+F5**. Express serves static with `maxAge: 1d`, so browser cache can stick.
- Check the terminal for EJS errors — usually a typo in a template variable.

Cart is wrong / stuck

The cart lives in your browser's localStorage. Clear it from DevTools → Application → Local Storage → delete the `jrs-cart-v1` key. Or in the browser console:

```
localStorage.removeItem('jrs-cart-v1'); location.reload()
```

Edits to products.json don't show up

Products are loaded once at boot. Save any watched file (`app.js` works) or stop & restart the server.

404 on an image I added

- Path is case-sensitive on macOS/Linux even if it works on Windows. Match the filename exactly.
- The path in `products.json` is the URL path, e.g. `/images/tees/foo.jpg`, not a Windows path.

- File must live under `public/` — that's the only directory Express serves statically.

8. NEXT-STEPS ROADMAP

Ordered roughly by ‘what unlocks selling first.’ Treat each phase as a stand-alone branch / PR.

Phase 1 · Make it shoppable

Goal: a real customer can buy a tee. ~1–2 days of focused work.

- **Real product photos.** Replace the SVG placeholders. Single biggest visual upgrade.
- **Stripe Checkout (hosted)** — `npm.cmd install stripe`. Add `POST /api/checkout` that builds a Stripe Checkout session from the localStorage cart payload, returns the session URL. Stripe handles the entire payment UI — no PCI scope for you.
- **Webhook for fulfilment** — `POST /api/webhook` listens for `checkout.session.completed` and emails you the order. Use `stripe-cli` locally to forward events.
- **Success / cancel pages** — `/order/success` and `/order/cancel`. Clear the localStorage cart on success.
- **Inventory decrement** — on webhook, subtract from `products.json` stock. Crude but enough until phase 2.
- **Shipping & tax** — configure inside Stripe Checkout (Stripe Tax + shipping rates).

Phase 2 · Real data layer

Goal: stop editing JSON by hand; orders persist.

- **SQLite + better-sqlite3** — smallest possible jump. Tables: `products`, `orders`, `order_items`.
- **Migration script** — import the JSON into the DB on first run.
- **Repository module** — wrap DB calls so routes don't touch SQL directly. Makes a future Postgres swap easy.
- **Real stock tracking** per size — expand the schema to `product_variants(product_id, size, stock)`.

Phase 3 · Admin GUI

Goal: edit products from a browser; no SSH-and-vim.

- **Auth.** Start with HTTP basic auth + a single `ADMIN_PASSWORD` env var. Upgrade to sessions when you add a second user.
- `/admin` — protected list view: products with edit / delete / ‘mark out of stock.’

- `/admin/products/new` — form to create products. Image upload to `public/images/tees/`.
- `/admin/orders` — list orders pulled from the DB / Stripe.

Phase 4 · Production readiness

Goal: ship it to the public internet.

- **Hosting.** Easiest path: Fly.io, Railway, or Render. They all do Dockerless Node deploys with auto HTTPS.
- **Env vars in prod** — `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `SITE_URL`, `ADMIN_PASSWORD`, `SESSION_SECRET`.
- **Logging.** Add `morgan` for request logs. Pipe to the host's log drain.
- **Security headers.** Add `helmet`. Set a CSP that allows fonts.googleapis.com and Stripe.
- **Rate limit** the checkout + admin endpoints with `express-rate-limit`.
- **Health check.** `GET /healthz` returning 200. Most hosts require one.
- **Backups.** SQLite: nightly copy of the DB file to S3 / R2.
- **Analytics.** Plausible or Fathom (privacy-friendly, no cookie banner).

Phase 5 · Polish & growth

- **Email.** Order confirmation + shipping notification via Resend or Postmark.
- **Newsletter capture.** Footer email signup → ConvertKit / Buttondown.
- **Customer accounts.** Order history, saved addresses. Pull this in only when customers ask.
- **Wishlist.** localStorage-first, sync to account when logged in.
- **Sizing chart modal** on product pages.
- **Reviews** — start with manual curation (a few testimonials in EJS) before adding a real reviews system.
- **i18n / multi-currency.** Only when you actually have international demand.

> **Heuristic for what to build next**
>
> Whatever blocks the next sale wins. Photos beat features; a working Stripe checkout beats a fancy admin GUI; a stable site beats a clever site. Defer everything that's not on that critical path until phase 5.